A Digital Twin-Enabled Framework for Automated Structural Clash Detection and Load Path-Aware Severity Classification in BIM Using Python and Revit API

Maleeha Waseem[a,*], Muhammad Arham[a], Muhammed Ahmed[a]

[a] *[Department, University, City, Country]*

[*] Corresponding author. E-mail addresses: maleehawaseem30@gmail.com (M. Waseem), muhammadarham6333@gmail.com (M. Arham), muhammed.ahmed0999@gmail.com (M. Ahmed).

Abstract

Clash detection reports thousands of interferences in building information models but cannot tell which ones matter structurally. This paper presents a digital twin-enabled framework that extracts structural and MEP elements through the Revit API with Python, detects clashes on exact meshes, builds a gravity load-path graph that conserves load exactly, and grades each clash by combining a code-informed local capacity-loss index with the load that depends on the member. Revit change events update the analysis incrementally, and results are written back. On two federated models, detection agreed with Revit's solid-intersection filter (F1 0.994 and 0.978). Geometry-only triage prioritised 49 % of 1,025 hard clashes; the proposed method prioritised 2.6 %, including flange and chord cuts that geometry rated minor. A held-out building exposed, and led to correcting, structural-system assumptions. Incremental updates took 37–62 ms for MEP and 2.5–5.6 s for structural changes, identical to full recomputation.

Keywords: Building information modelling; Clash detection; Digital twin; Load path; Severity classification; Revit API; Structural robustness

1 Introduction

Clash detection is one of the most widely used building information modelling (BIM) workflows. Federated discipline models are checked geometrically and every interference between, for example, a duct and a beam is reported for resolution before construction. The economic case is well documented: unresolved design clashes that surface on site lead to rework, delay and cost [1], and coordination problems remain frequent because disciplines model in isolation [2]. Yet the output of commercial clash engines is notoriously noisy. Thousands of interferences are reported for a single building, and a large share of them are irrelevant, duplicates of one design issue, or resolvable later without consequence [3,4]. Coordinators therefore spend much of their time triaging reports rather than resolving problems [5,6].

The research response has concentrated on *relevance*: learning, from historical coordination decisions, which clashes a human coordinator would discard [3,4,6,7], or ranking clashes with expert-weighted attributes of the elements involved [8]. These approaches reproduce coordinator judgement, but they do not reason about what a clash does to the structure. For structural members this is the question that matters. A 100 mm pipe through the web of a girder near its support, through the tension zone of a beam at mid-span, or through a slab within the punching-shear perimeter of a column can be far more serious than a much larger duct that merely grazes a non-load-bearing element. Two clashes with identical geometry can also have very different consequences, depending on how much of the building depends on the member that is cut. That dependency is a property of the structure's *load path*, which is not represented in any clash report.

At the same time, BIM practice is moving from periodic, file-based coordination toward *digital twins*: continuously synchronised virtual representations that are updated as the design or the asset changes [9–12]. In a twin, clash checking should not be a batch job run once a

week. It should be an incremental service that re-evaluates only what changed and pushes its assessment back to the authoring environment. Most clash-management research, however, still assumes a static export to a separate tool.

This paper addresses both gaps with a framework that (i) extracts structural and MEP elements, their geometry and their connectivity directly from Autodesk Revit through its application programming interface (API) using Python; (ii) detects hard, touch and clearance clashes with exact mesh geometry; (iii) builds a gravity *load-path graph* of the structure and derives, for every member, the load that depends on it and the load that has no alternative path if it fails; (iv) classifies each clash by combining a code-informed estimate of the *local* capacity loss with this *global* load-path consequence; and (v) keeps all of this synchronised with the authoring model as a digital twin that processes Revit change events incrementally and writes its results back into Revit. The specific contributions are:

1. A load-path graph built automatically from BIM geometry and Revit connectivity, with load propagation by tributary areas, exact equilibrium, and a dominator-tree measure of *orphaned load* that operationalises the alternate-load-path concept used in progressive-collapse design [13,14].
2. A two-axis severity model, a demand-weighted capacity loss index (local) combined with a load-path consequence expressed in bay-equivalents (global), that is transparent, member-type specific and adapted from design provisions for openings in concrete and steel members [15,16].
3. A digital-twin synchronisation layer with fingerprint-based change detection and incremental clash, load-path and severity updates that are provably identical to full recomputation, closed through a pyRevit hook and a write-back command.
4. An open, reproducible evaluation on two federated Revit models of different structural systems: detection accuracy against Revit's own solid-intersection filter, load-path

equilibrium, comparison with geometry-only and category-based triage, a one-at-a-time sensitivity analysis and twin latency. It also includes a held-out test that exposed, and led to correcting, two rules that silently assumed a particular structural system.

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 presents the framework. Section 4 describes the implementation, the case-study models and the evaluation design. Section 5 reports the results, Section 6 discusses implications and limitations, and Section 7 concludes.

2 Related work

2.1 Automated clash detection and relevance filtering

Clash detection engines compare element geometry pairwise after a spatial pre-selection. Bounding-volume hierarchies of axis-aligned [17] or oriented [18] boxes and spatial indexes such as the R-tree [19] keep the number of exact tests manageable, and general-purpose libraries such as FCL provide collision, distance and proximity queries on triangle meshes [20]. In practice the bottleneck is not computation but interpretation. Hu and Castro-Lacouture trained supervised classifiers to separate relevant from irrelevant clashes using attributes of the clashing components [3], and later exploited dependencies between components and between clashes to improve detection holistically and to optimise correction sequences [21–23]. Lin and Huang combined expert rules with supervised learning to filter irrelevant clashes [4]. More recent work packages relevance filtering as a plug-in within the BIM environment [7], introduces risk and relevance analysis aligned with how coordinators work [6], proposes AI-based triage that also groups recurring clashes [5], and prioritises structure–MEP hard clashes with expert-weighted criteria derived by the Best–Worst Method [8]. Knowledge-based [24,25] and learning-based [26] approaches have also been proposed to

support clash *resolution*, and recent work links MEP–structural coordination to on-site execution through path planning and augmented reality [27].

These studies share two characteristics. Severity or relevance is inferred from element *attributes* (type, discipline, size, location) or learned from past decisions, and the structural role of the clashing member is at most a categorical feature. None of them, to the authors' knowledge, evaluates how much capacity a penetration removes at its position along the member, or how much of the building depends on that member.

2.2 Rule-based checking and structural knowledge in BIM

Automated rule checking translates code requirements into executable checks on BIM data [28,29], with applications that range from construction safety [30] to MEP systems [31]. Structural knowledge has been brought into BIM chiefly through analytical models, whose interpretation from the coordination view is itself non-trivial [32]. Design codes and guides do contain explicit provisions for penetrations: ACI 318 limits openings in two-way slabs by their position relative to column and middle strips, reduces the punching-shear perimeter for openings within four slab thicknesses of a column, and limits conduits embedded in slabs [15]. Steel design guidance limits web openings in depth and location [16,33], and experimental work confirms that the behaviour of concrete beams with web openings depends strongly on the opening's size and its position relative to supports and the section depth [34–36]. These provisions have not, so far, been used to grade clashes automatically.

2.3 Digital twins in design and construction

The digital-twin concept describes a virtual counterpart that is linked to its physical system through continuous data exchange [9]. Kritzinger et al. distinguish digital models (manual exchange), digital shadows (automatic one-way exchange) and digital twins (automatic exchange in both directions) [37]. In construction, reviews document rapid growth together

with ambiguity in how the term is used [12,38,39]. Boje et al. argue for semantically rich construction digital twins that go beyond static BIM [10], Sacks et al. frame digital twin information systems around closed control loops between the as-designed and as-built states [11], and BIM-enabled twins have been demonstrated for real-time structural health monitoring [40]. Design coordination, the stage at which clashes arise, has received less attention: clash checking is usually run on exported snapshots, and its results are not fed back into the authoring model automatically.

2.4 Load paths and structural robustness

A load path is the route by which loads travel from where they are applied to the foundations. Robustness, the insensitivity of a structure to local damage, depends on whether alternative paths exist when a member is lost [14,41]. The alternate path method of progressive-collapse design removes members notionally and checks whether the remaining structure can bridge the damage [13,42]. Load-transfer indices such as U* quantify load paths from finite-element responses [43], but they require an analysis model. Graph representations are a natural, lightweight way to encode support relations. Dominance, a classical graph concept in which node d dominates node n if every path from the root to n passes through d [44], maps directly onto the question "whose failure leaves this load with no other path to the ground?".

2.5 Research gap

The literature therefore offers (i) fast and accurate geometric detection, (ii) learning- and rule-based relevance filtering driven by element attributes, (iii) code provisions for penetrations that are applied manually, and (iv) digital-twin architectures that rarely address design coordination. What is missing is a method that grades structural clashes by *mechanics* (the capacity a penetration removes where it occurs) and by *consequence* (the load that depends on

the member), computed automatically from the BIM authoring model and kept up to date as the model changes. This study aims to fill that gap and asks four research questions:

- **RQ1** Can structural clashes be detected from Revit API data with accuracy comparable to Revit's own geometric engine, and fast enough for interactive use?
- **RQ2** Can a gravity load-path graph that satisfies equilibrium be constructed automatically from BIM geometry and connectivity?
- **RQ3** How does load-path-aware severity classification differ from geometry-only and category-based triage, and how sensitive is it to its judgement-based parameters?
- **RQ4** Can the analysis be maintained incrementally as a digital twin, with results identical to full recomputation, at interactive latency?

3 Framework

3.1 Overview

Fig. 1 shows the architecture. A pyRevit extension running inside Revit extracts structural and MEP elements from the host model and all loaded links (Section 3.2). A Python analysis service detects clashes (Section 3.3), builds the load-path graph (Section 3.4) and classifies severity (Section 3.5). The digital-twin layer keeps the service synchronised with Revit through change events and writes results back (Section 3.6). The two sides exchange JSON documents through a file-based inbox and outbox, so the analysis runs in a standard CPython environment with scientific libraries, independent of Revit's embedded interpreter.

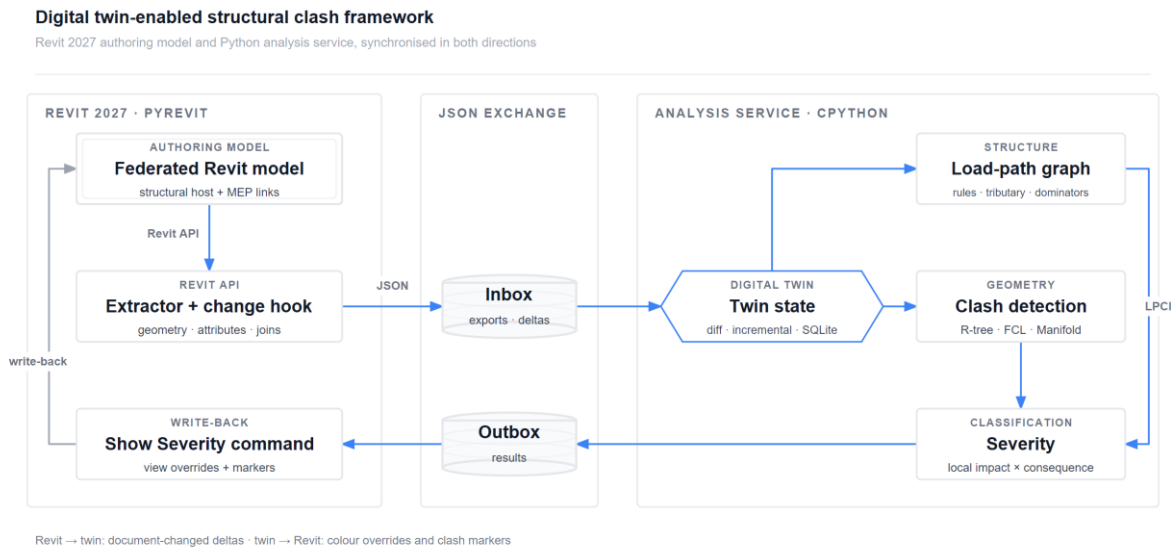


Fig. 1. Architecture of the digital twin-enabled framework. Revit-side components (left) run inside Revit through pyRevit; the analysis service (right) runs in CPython. The loop is closed by the document-changed hook (Revit → twin) and the write-back command (twin → Revit).

3.2 Data extraction through the Revit API

Scope. Structural elements are taken from the categories *Structural Columns*, *Structural Framing*, *Structural Foundations*, and from *Floors* and *Walls* whose *Structural* flag is set. MEP elements are ducts, pipes, cable trays, conduits, their fittings and accessories, flexible ducts and pipes, mechanical equipment and sprinklers. The host model and every loaded link (`RevitLinkInstance`) are traversed. By default, structural elements are taken only from the host, the structural model, so that floors or walls flagged as structural in an architectural link are not counted twice.

Geometry. For each element the solids of its fine-detail geometry, including instance geometry of families, are tessellated with `SolidUtils.TessellateSolidOrShell`. Vertices are transformed by the link's total transform into host coordinates, converted to metres and de-duplicated. Each shell component is a closed triangulation, so the meshes are watertight and support exact Boolean operations.

Attributes and connectivity. The extractor records category, family and type, level, location curve or point, bounding box, structural type, usage and material, section parameters (height,

width, web and flange thickness where available), floor and wall thickness, volume, and for MEP elements nominal and outer diameter, width, height, insulation thickness and system classification. Two kinds of Revit connectivity are stored for structural members: elements joined at each end of framing members (`LocationCurve.ElementsAtJoin`) and geometry joins (`JoinGeometryUtils`).

Reference intersections. To obtain an independent reference for detection accuracy, the extractor also asks Revit itself which MEP elements intersect each structural solid. The solid is transformed into the MEP link's coordinate system and filtered with `BoundingBoxIntersectsFilter` followed by Revit's exact `ElementIntersectsSolidFilter`.

Batch and interactive use. The same library serves a headless batch script, run with `pyrevit run` and opening models detached and read-only, and the interactive ribbon commands described in Section 3.6.

3.3 Clash detection

Let $\mathcal{S}$ and $\mathcal{M}$ be the structural and MEP element sets. A clash is a pair $(s, m) \in \mathcal{S} \times \mathcal{M}$ whose meshes intersect or come closer than a clearance c (25 mm by default).

Broad phase. MEP bounding boxes are inflated by c and inserted into an R-tree [19]. Each structural bounding box is queried against it, giving the candidate set. A vectorised brute-force test over all $|\mathcal{S}|\,|\mathcal{M}|$ pairs serves as the baseline and as a correctness check; both must return identical candidate sets.

Narrow phase. Each candidate pair is tested with FCL bounding-volume-hierarchy collision on the triangle meshes [20]. Surface-triangle tests cannot see *containment*, for example an elbow embedded entirely within a slab, so when no surface crossing is found the framework also tests whether vertices of either mesh lie inside the other. This uses a deterministic ray-parity test along three fixed directions. Colliding or contained pairs are intersected with a

robust mesh Boolean (Manifold), which gives the intersection volume V and the intersection solid. Families built from several solids, such as the bars of an open-web joist, are intersected component by component. Components that are not closed, typically parts of detailed equipment families, are represented by their convex hulls, a deterministic and slightly conservative stand-in. A point-sampling estimate is used only if the Boolean fails. A pair is a *hard* clash if $V > 1\ \mathrm{cm}^3$ and a *touch* otherwise. Non-colliding pairs are submitted to an FCL distance query and recorded as *clearance* clashes if $0 < d \leq c$.

Localisation. Every structural member is given a local frame $(\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w})$: $\boldsymbol{u}$ along its location curve (vertical for columns, the principal in-plan direction for slabs), $\boldsymbol{w}$ vertical for horizontal members, and $\boldsymbol{v} = \boldsymbol{w} \times \boldsymbol{u}$. Its length L, width b and depth h are measured from its own mesh. This makes the analysis independent of family-specific parameter names. The intersection solid is expressed in this frame, giving its extents $(\Delta u, \Delta v, \Delta w)$, its ranges within the cross-section, and its normalised position along the member, $\xi \in [0,1]$. The angle between the MEP and member axes and whether the MEP run is vertical are also recorded.

3.4 Gravity load-path graph

The load-path graph $G = (V, E)$ has one node per structural member plus a virtual node Γ (ground). A directed edge $a \to b$ means "a is supported by b", so load flows along the edges.

Contacts. Candidate support relations are collected from three sources: (i) *physical contact*, a mesh distance of at most 50 mm (FCL); (ii) *Revit joins* exported with the elements; and (iii) *member-end proximity*, where a beam or brace end, taken from its location curve, lies within 150 mm of another member. The third source mirrors how analytical models connect members by centre lines. It is necessary because the physical geometry of steel members is often cut back at connections, leaving the mesh short of its support (Section 5.2).

Orientation. Each contact is oriented, or discarded, by engineering rules (Table 1). They encode the gravity hierarchy slab → beam → girder → column/wall → foundation → ground,

together with transfer conditions. Between a beam and a column or wall, the decisive question is whether the vertical member *continues below* the beam: if it does, the beam frames into it; if it starts within or above the beam's depth and rises above it, it stands on the beam (transfer). This formulation is insensitive to timber joinery, where tenons sink into sill beams. Braces belong to the lateral system; in the gravity graph they hang on the members they connect and never act as supports. Columns and walls separated only by a slab are connected directly (stacking), so that a column's load is not routed through the floor plate. Foundations, and members with no support whose base lies at or below the foundation level, are connected to Γ. Any directed cycle is broken by removing the edge that most violates the gravity hierarchy, and members with no path to Γ are reported as *unsupported*.

Table 1. Orientation rules for a contact between members a and b (z^{bot}, z^{top}: bottom and top elevations; t: contact tolerance).

Member pair	Rule (edge = "is supported by")
slab – beam	slab → beam if $z_{beam}^{\mathrm{top}} \leq z_{slab}^{\mathrm{top}} + t$, else beam → slab
beam – column / wall	beam → column/wall if the vertical member continues below the beam ($z_v^{\mathrm{bot}} < z_{beam}^{\mathrm{bot}} - t$); column/wall → beam (transfer) if it starts within or above the beam's depth and rises above it ($z_v^{\mathrm{top}} > z_{beam}^{\mathrm{top}} + t$); a stub within the beam depth: beam → member if a beam end lies within its half-width $+t + 0.15$ m
brace – any	brace → member (braces never support); brace – brace: no gravity relation
beam – beam	secondary → primary if one beam's end lands on the other's span (not end-to-end)
column/wall – column/wall	upper → lower (stacking, also across a slab if the gap is ≤ 1.2 m and plans overlap ≥ 25 %)
column/wall – slab	slab → column/wall, unless the member stands on the slab (transfer)
any – foundation	member → foundation; foundation → ground Γ
slab – slab, foundation – foundation	no gravity relation

Loads and propagation. Each member contributes a factored self-weight $q_i^{\mathrm{own}} = 1.2\,\gamma\,V_i$, with $\gamma = 24$ kN/m³ for concrete and 77 kN/m³ for steel. Slabs additionally carry superimposed dead and live loads (default 1.5 and 2.4 kPa, the latter for offices [45]) in the combination $1.2D + 1.6L$. Loads are propagated in topological order: member i carries $D_i = q_i^{\mathrm{own}} + \sum_{j \to i} \phi_{ji}\ D_j$ and passes it to its supports with fractions ϕ_{ik} that sum to one. For slabs,

ϕ is obtained by the tributary-area method: a 0.5 m plan grid on the slab is assigned to the nearest support, and equidistant points are shared equally. For beams, braces and walls, it is the one-dimensional tributary length between the projections of the supports onto the member axis. For other members it is proportional to plan overlap. Load is sent only to supports that themselves reach Γ, so the load entering the ground equals the total load of all supported members exactly.

Orphaned load. In the reversed graph rooted at Γ, the immediate-dominator tree [44,46] identifies, for each member i, the set $\mathrm{Dom}(i)$ of members whose *every* path to the ground passes through i. The orphaned load

$$E_i = \sum_{j \in \mathrm{Dom}(i)} q_j^{\mathrm{own}} \qquad (1)$$

is the load that would be left without any alternative path if i were lost. It is a graph analogue of the alternate-path check of progressive-collapse design [13,14]: a member with $E_i = 0$ is redundant in the gravity system, whereas a transfer girder carrying columns has a large E_i.

Load-path criticality index. The dependent load D_i and the orphaned load E_i are combined into

$$\mathrm{LPCI}_i = (1-\alpha)\, g(D_i) + \alpha\, g(E_i), \qquad g(x) = \frac{\ln(1 + x/D_{\mathrm{ref}})}{\ln(1 + D_{\max}/D_{\mathrm{ref}})} \qquad (2)$$

where D_{ref} is the median load that a member receives *directly from a slab it supports*, i.e. a typical floor bay whether the slab bears on joists, beams, walls or columns; $D_{\max}$ is the largest load carried by any non-slab member; and $\alpha = 0.3$. The logarithm places members whose loads differ by orders of magnitude (a joist, a girder, a ground-floor column) on one comparable scale. Because a whole storey slab is often a single Revit element, the consequence of an opening in a slab is evaluated for the *local panel* instead: $D = q\,R^2$ with $R = \mathrm{clip}(2d_s, 1{,}6)$ m, where d_s is the plan distance from the clash to the nearest member beneath the slab, so that a point midway between supports s apart gives a panel of side s. To

this is added the load of any column or wall bearing on the slab within R. For a slab on grade, $R = 1$ m.

3.5 Load path-aware severity classification

Severity combines a *local* impact, the capacity removed at the clash location, with a *global* consequence, the load that depends on the member there.

Local impact. The demand-weighted capacity loss index is

$$\eta = \max_{a} f_a(\xi)\, \ell_a \qquad (3)$$

where, for each action a (flexure, shear, axial, punching, …), $\ell_a \in [0,1]$ is the fraction of the corresponding sectional capacity removed by the clash region and $f_a(\xi) \in [0,1]$ is the relative demand of that action at the clash location. For beams, the simply-supported shapes $f_M = 4\xi(1-\xi)$ and $f_V = |1-2\xi|$ are used, so a penetration near a support is governed by shear and one near mid-span by flexure. The loss models, given in full in the released code, are:

- *Concrete beams*: a rectangular hole of height h_o and width b_h at offset y_0 from mid-depth reduces the second moment of area by $\ell_M = (b_h/b)\,[(h_o/h)^3 + 12(h_o/h)(y_0/h)^2]$. The shear loss is $\ell_V = (b_h/b)(h_o/h)(1 + 0.5\,\Delta u/h)$, so that long openings, which sever more stirrups, cost more. An opening reaching the outer 50 mm, where longitudinal bars lie, has $\ell_M \geq 0.5\, b_h/b$. Openings deeper than $h/3$ through more than half the width are "large openings" [34] with $\ell \geq 0.15$.
- *Steel I-sections*: the lost plastic modulus is summed over flanges and web from the intersection's exact position in the section, $\ell_M = Z_{\text{lost}}/Z$, and the shear loss is the fraction of web depth removed. Web openings within one depth of a support and deeper than $0.7d$ are flagged after AISC Design Guide 2 [16].
- *Open-web joists and trusses* (identified by family name or a solid fill ratio below 8 %): ℓ is the fraction of the joist material within the clash slice that the MEP element

occupies, attributed to the chords (flexure) or the web members (shear) by its position in the depth.

- *Columns and braces*: ℓ_N is the fraction of cross-sectional area removed, with $f_N = 1$.
- *Walls*: the fraction of wall length interrupted, weighted by 1.0 within the boundary zone near the wall ends and 0.6 elsewhere.
- *Slabs*: a horizontal MEP run inside the slab is an *embedded* item, with $\ell = 0.375\, D_{\mathrm{MEP}}/t$, so that $\eta = 0.125$ at ACI's limit of $t/3$ [15]. A penetration is assessed for punching, as the fraction of the critical perimeter b_0 shadowed when it lies within $4t$ of a column face, and for two-way flexure with a demand factor of 1.0, 0.5 or 0.1 in the column-strip/column-strip, column-strip/middle-strip and middle-strip/middle-strip zones, following the strip-based allowances of ACI 318 §8.5.4. Slabs supported on beams or walls are treated in one-way action relative to the local panel width.
- *Foundations*: the fraction of plan area removed.

Clearance clashes and touches have $\eta = 0$.

Consequence levels. The dependent and orphaned loads at the clash location are expressed in *bay-equivalents*, $n_D = D/D_{\mathrm{ref}}$ and $n_E = E/D_{\mathrm{ref}}$. The consequence level is C3 (global) if $n_D \geq 15$ or $n_E \geq 5$, C2 (extended) if $n_D \geq 3$ or $n_E \geq 1$, and C1 (local) otherwise.

Classification and ranking. The impact level (L0: $\eta < 0.05$; L1: < 0.15; L2: < 0.35; L3: ≥ 0.35) and the consequence level index a risk matrix (Table 2) that returns *Negligible*, *Minor*, *Major* or *Critical*. Clearance clashes are classified separately as *Coordination*. For ranking within and across classes, the continuous score

$$s = \eta\, \frac{1 + \beta\, \mathrm{LPCI}}{1 + \beta}, \qquad \beta = 2, \qquad (4)$$

is used, so that equal local damage ranks higher on a more critical member.

Table 2. Severity matrix (impact level × consequence level).

Impact \ Consequence	C1 local	C2 extended	C3 global

Impact \ Consequence	C1 local	C2 extended	C3 global
L0 ($\eta < 0.05$)	Negligible	Negligible	Minor
L1 ($0.05 \leq \eta < 0.15$)	Minor	Minor	Major
L2 ($0.15 \leq \eta < 0.35$)	Minor	Major	Critical
L3 ($\eta \geq 0.35$)	Major	Critical	Critical

The thresholds are judgement-based, so they are the subject of the sensitivity analysis in Section 5.4.

3.6 Digital-twin synchronisation

State and versioning. The twin keeps the current element set, an R-tree of MEP boxes, the cached contact pairs of the load-path graph, the clash set and the assessments, and records every version in SQLite: element fingerprints, the clash set with severities, and an event log. An element's fingerprint is a SHA-1 hash of its rounded mesh, its properties and its category.

Revit → twin. A pyRevit `doc-changed` hook receives Revit's `DocumentChanged` event, which lists the added, modified and deleted element ids of each committed transaction. It serialises the affected structural and MEP elements with the same extractor and drops the result into the twin's inbox; it is active only while synchronisation is switched on. A full export can be triggered from the ribbon at any time.

Incremental update. For each delta, the twin (i) diffs fingerprints into added, modified and deleted sets; (ii) removes the clashes that involve them; (iii) re-runs the narrow phase only for *changed structural* × *all MEP*, through the R-tree, and *all structural* × *changed MEP*; (iv) if any structural member changed, drops its cached contacts, recomputes contacts for the changed members only, and re-orients, re-propagates and re-scores the graph, which are inexpensive operations; and (v) re-assesses the affected clashes, or all clashes after a structural change, because support sets can move. The result is designed to be identical to a full recomputation of the same model state, and this property is tested (Section 5.5).

Twin → Revit. A *Show Severity* command reads the latest results, colours each clashing structural member in the active view by its worst severity, and places a marker (a

`DirectShape`) at every Critical and Major clash. The marker's comments record the class, η, LPCI and the element ids. In the classification of Kritzinger et al. [37], the coupling between the authoring model and the analysis is therefore automatic in both directions. The coupling to the *physical* asset is discussed in Section 6.2.

4 Implementation and evaluation design

4.1 Implementation

The Revit side is a pyRevit [47] extension for Revit 2027 (build 27.0.4, .NET 10), written for its IronPython 2.7 engine, with three ribbon commands (*Export Twin*, *Twin Sync*, *Show Severity*) and a `doc-changed` hook. API usage was checked against a reference generated from the installed `RevitAPI.dll` and `RevitAPIUI.dll` [48]. The analysis service is a Python 3.11 package. It uses trimesh and Manifold for meshes and Booleans, python-fcl [20] for collision and distance, an R-tree [19] for spatial indexing, NetworkX [46] for graphs and dominators, and SQLite for twin versions. All experiments were run on one Windows 11 workstation, with Revit and the analysis service on the same machine. Synthetic verification cases with known answers are part of an automated test suite: a one-bay frame with hand-checkable tributary loads, a pipe through a beam whose intersection volume is known analytically, a fully embedded conduit, and incremental-versus-full equivalence checks for the twin.

A generative AI coding agent, Claude Code with the Claude Opus 5.5 model (Anthropic), assisted with software development and data analysis. Under the authors' direction it wrote the Revit extension, the analysis package, the experiment scripts and the scripts that generate the figures and tables, and it ran the experiments. The architecture diagram (Fig. 1) was rendered from a layout specification written with the same tool. Its output was verified independently of the agent in four ways: the automated test suite described above, agreement

with Revit's own solid-intersection filter (Section 5.1), exact load equilibrium in both models, and identity of incremental and full recomputation in the twin benchmark. Every number, table and plot in this paper is generated by the released scripts from the stored results, so all of them can be reproduced. The agent also searched the literature through the OpenAlex API, and every reference was checked against its DOI record before inclusion.

4.2 Case-study models

Two federated models distributed with Revit 2027 were used (Table 3). *Snowdon Towers* is a multi-storey building with separate structural, architectural, HVAC, plumbing, electrical and site models. Its structure combines steel W-shape girders, open-web steel joists (K- and LH-series), concrete beams, columns, walls and slabs on spread footings. *Golden Nugget* consists of an architectural-and-structural model and a building-services model. The structural model of each building was opened as host with its MEP models as links, so all elements were extracted in one coordinate system. Both models were opened detached and read-only, and no element was modified except in the scripted twin scenarios, which were never saved.

Table 3. Case-study models (Revit 2027 sample files). Structural members are taken from the host (structural) model; MEP elements from the host and its links.

Model	Structural members	Beams / braces / columns / walls / slabs / foundations	MEP elements	MEP by source model
Snowdon Towers	1,185	959 / 0 / 54 / 58 / 24 / 90	9,129	Snowdon Towers Sample HVAC (2,118); Snowdon Towers Sample Electrical (852); Snowdon Towers Sample Plumbing (6,159)
Golden Nugget	637	277 / 59 / 139 / 102 / 45 / 15	5,297	Architektur und Ingenieurbau (14); Gebaeudetechnik (5,283)

Snowdon Towers served as the *development* case: errors found while analysing it led to the corrections described in Sections 5.1 and 5.2 (containment, connection set-backs, open-web members, component-wise Booleans). The method was then frozen and applied unchanged to Golden Nugget as a *held-out* case. The held-out run exposed assumptions that did not generalise and one reproducibility defect. The following were therefore revised and re-applied to **both** models: (i) the reference bay load D_{ref}; (ii) the beam–column and brace orientation

rules; (iii) the reinforcement-zone rule, now restricted to concrete; and (iv) the handling of open components (convex hulls and a deterministic inside test instead of random sampling). Section 5.6 reports the frozen and revised results side by side, and all other results use the revised method. In the development case, items (i) and (ii) changed the classification of three clashes and items (iii) and (iv) changed none.

4.3 Evaluation design

- **RQ1, detection.** Pairs classified as physical clashes (hard or touch) were compared with the pairs returned by Revit's `ElementIntersectsSolidFilter` for the same elements, giving true positives, false positives and false negatives. Every disagreement was inspected. Broad-phase scalability was measured by tiling a model $k = 1,2,4,8$ times and timing the brute-force and R-tree broad phases, whose candidate sets were asserted to be identical.
- **RQ2, load path.** Global equilibrium (load entering the ground versus the total load of supported members), the number of unsupported members and their causes, and the distribution of carried loads by member type were examined.
- **RQ3, severity.** The proposed classification was compared with three alternatives: (i) *local impact only*, the same η with every member assigned the middle consequence level (an ablation of the load path); (ii) *geometry only*, a class from the smallest clash extent with thresholds of 10, 50 and 150 mm, reflecting penetration-depth triage in common clash software; and (iii) *category rules*, a priority by member type (column > beam, wall, foundation > slab) raised by one level for MEP elements of 300 mm or more. A one-at-a-time sensitivity analysis varied the impact and consequence thresholds (×0.5 to ×2), α, β, the live and superimposed dead loads, and the contact and end tolerances. It reports the Jaccard overlap of the Critical+Major set, the

quadratic-weighted Cohen's κ of the four classes, and Kendall's τ of the top-50 scores with respect to the reference configuration.

- **RQ4, digital twin.** A seeded stream of 120 design changes (20 each of MEP move, resize, add and delete, and beam move and delete) was applied one at a time through the twin. After every 20 events, the incremental state was compared with a full recomputation of the same model state, and the full recomputation was timed.

5 Results

5.1 Clash detection (RQ1)

In Snowdon Towers, the extractor exported 1,185 structural members (959 beams, 54 columns, 90 foundations, 58 structural walls and 24 slabs) and 9,129 MEP elements from three links (HVAC 2,118; electrical 852; plumbing 6,159). It found 687 hard clashes, 17 touches and 592 clearance clashes (Table 4). Of the hard clashes, 602 involved slabs (mostly pipes and conduits through floors), 43 beams, 37 walls and 5 foundations. Clearance clashes concentrated on beams (478), where ducts and pipes run just beneath the framing.

Table 4. Clash detection compared with Revit's ElementIntersectsSolidFilter. Physical clashes = hard + touch; TP / FP / FN are pairs agreed / found only by the framework / found only by Revit.

Quantity	Snowdon Towers	Golden Nugget
Candidate pairs (broad phase)	2,040	3,724
Clashes: hard / touch / clearance	687 / 17 / 592	338 / 31 / 715
Revit reference pairs	697	357
TP / FP / FN	696 / 8 / 1	355 / 14 / 2
Precision	0.989	0.962
Recall	0.999	0.994
F1	0.994	0.978
Time, framework broad + narrow phase (s)	17	46
Time, Revit reference filter (s)	544	32

Agreement with Revit's own solid-intersection filter was high: 696 of Revit's 697 pairs were also found (recall 0.999), and 8 of the 704 physical clashes were not in Revit's set (precision 0.989). All 8 are zero-volume face contacts (touches), which Revit's filter excludes by definition, and the single remaining miss is a pipe whose polygonal tessellation stops short of a slab face. Two errors found during development explain why exact geometry handling

matters. First, mesh–mesh collision tests only surface triangles, so 59 MEP elements lying *entirely inside* slabs or walls (embedded elbows, pipes inside walls) were invisible to them, which initially lowered recall to 0.914. The containment test recovered them. Second, the open-web joists (482 members) are modelled as touching bar solids and are not watertight as single meshes. A sampling fallback underestimated their intersection volumes by up to a factor of 2.6 and was not reproducible run-to-run, whereas the component-wise exact Boolean is both exact and deterministic. After these corrections, all 704 physical clashes are measured by an exact Boolean, one of them with a convex-hull stand-in for an open component.

Run times are shown in Fig. 2. The R-tree reduced the 10.8 million structural–MEP pairs to 2,040 candidates in 0.09 s. When the model was tiled up to eight times (692 million pairs), the brute-force broad phase grew quadratically to 45 s, whereas the R-tree grew almost linearly to 0.84 s (Fig. 2a). The complete pipeline (detection 17 s, load-path graph 17 s, severity 5 s) took 39 s, against 544 s for Revit's filter to produce its reference pairs for the same elements (Fig. 2b). That comparison does not generalise to every model (Section 5.6).

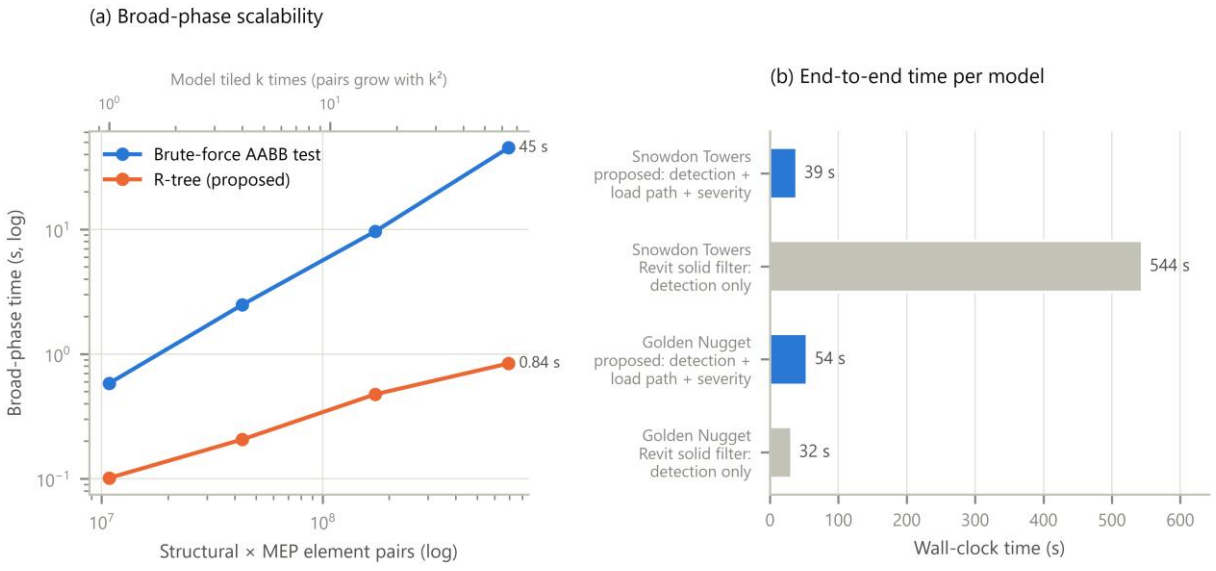


Fig. 2. Detection efficiency. (a) Broad-phase time against the number of structural–MEP element pairs when Snowdon Towers is tiled k = 1, 2, 4, 8 times; candidate sets are identical. (b) End-to-end time of the proposed pipeline and of Revit's ElementIntersectsSolidFilter.

5.2 Load-path graph (RQ2)

The graph of Snowdon Towers has 1,185 member nodes and 3,432 support edges, from 4,059 contact pairs. Only one directed cycle had to be broken. Fig. 3a shows the graph and Fig. 3b the distribution of carried loads. The reference bay load, the median load a member receives directly from a slab, is $D_{\mathrm{ref}} = 51$ kN. Beams, most of which are joists, cluster around it, whereas walls, columns and foundations carry one to two orders of magnitude more, up to 14.1 MN. The total factored load of the supported structure, 114.2 MN, enters the ground exactly (ratio 1.0000), which confirms that propagation conserves load.

Three findings were instructive. (i) With mesh contact alone, 60 members (5.1 %) had no path to the ground. Most were W18x55 girders whose steel geometry is cut back at the connections, leaving a gap to the column larger than the contact tolerance, with no Revit join recorded. Adding member-end proximity reduced the unsupported set to 21, and the revised beam–column rule (Section 5.6) correctly placed two short roof-level columns on the beams they stand on. (ii) The remaining 18 members (1.5 %), 17 short steel beams and channels and one wall, have no modelled support. Flagging such members is a useful by-product of the framework. (iii) Before load was restricted to supports that reach the ground, 5 % of the load "leaked" into the unsupported cluster.

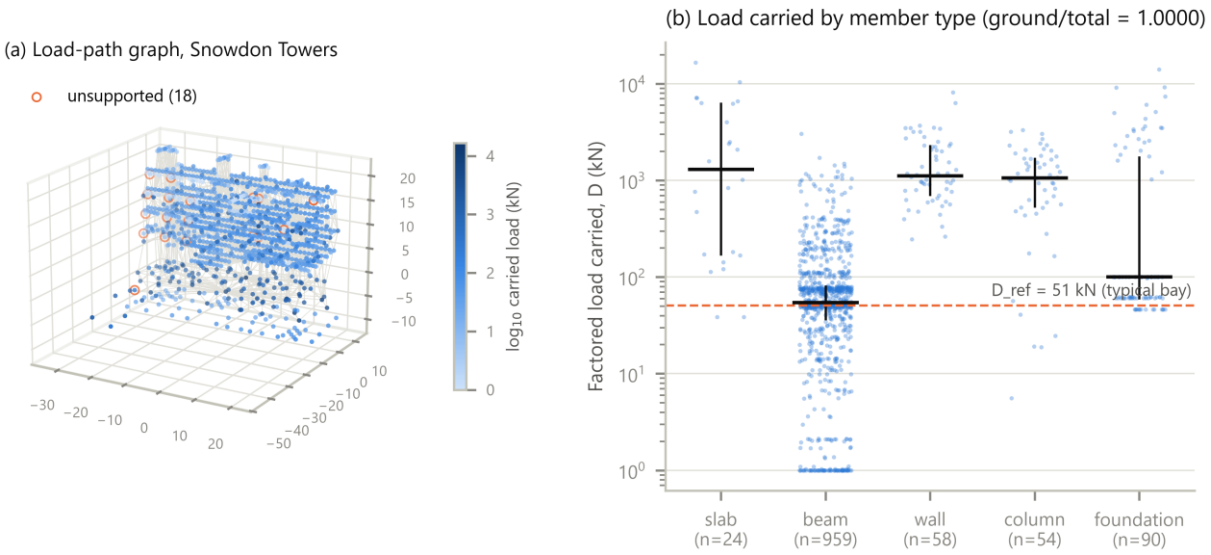


Fig. 3. Load-path graph of Snowdon Towers. (a) Members coloured by the factored load they carry (log scale); light grey lines are support edges; orange rings mark the 18 members with no path to the ground. (b) Carried load by member type; bars show the median and interquartile range; loads below 1 kN are shown at 1 kN.

5.3 Severity classification (RQ3)

Fig. 4 and Table 5 compare the four classifications of the 687 hard clashes in Snowdon Towers. The proposed method rated 1 clash Critical, 11 Major, 96 Minor and 579 Negligible. Geometry-only triage rated 43 Critical and 344 Major, so 387 clashes (56 %) would need priority attention. Category rules rated 4 Critical and 83 Major. Agreement with the proposed classes is close to chance for geometry-only triage (quadratic-weighted $\kappa = 0.03$) and low for category rules ($\kappa = 0.15$). The cross-tabulation (Fig. 5) shows that the methods disagree in *both* directions:

- Of the 387 clashes that geometry-only triage would prioritise, 349 are slab penetrations. These are predominantly vertical pipes passing through the floor plate, 426 of the slab clashes pass through the full thickness, and they would normally be resolved with a sleeve. The proposed method rated 320 of them Negligible and 62 Minor, because the opening is small relative to the column-strip width and the punching perimeter, and because the local panel load is small.

- Conversely, 7 of the 12 clashes that the proposed method rated Critical or Major were only *Minor* for geometry-only triage. Examples are a 203 mm duct clipping the bottom flange of a W-shape girder (Fig. 6b), where the penetration depth is small but the flange carries most of the flexural capacity, and mechanical units cutting the chords of open-web joists (Fig. 6c).

Table 5. Severity of the hard clashes under the four classification methods. κ_w: quadratic-weighted Cohen's kappa with the proposed classes; Jaccard: overlap of the Critical+Major sets with the proposed one.

Model	Method	Critical	Major	Minor	Negligible	Critical + Major	κ_w	Jaccard
Snowdon Towers	Proposed (load path-aware)	1	11	96	579	12 (1.7 %)	—	—
Snowdon Towers	Local impact only (ablation)	2	9	58	618	11 (1.6 %)	0.80	0.64
Snowdon Towers	Geometry only	43	344	286	14	387 (56.3 %)	0.03	0.01
Snowdon Towers	Category rules	4	83	600	0	87 (12.7 %)	0.15	0.12
Golden Nugget	Proposed (load path-aware)	0	15	126	197	15 (4.4 %)	—	—
Golden Nugget	Local impact only (ablation)	9	26	59	244	35 (10.4 %)	0.73	0.32
Golden Nugget	Geometry only	49	62	190	37	111 (32.8 %)	0.07	0.04
Golden Nugget	Category rules	62	201	75	0	263 (77.8 %)	0.06	0.05

The single Critical clash is a 273 mm pipe running along the inside of a foundation that carries about 5 MN, or 100 bay-equivalents (Fig. 6a). Its local impact alone ($\eta = 0.20$) would make it Major. The load-path ablation (*local impact only*) produced 11 Critical or Major clashes. Adding the consequence level downgraded two of them, chord cuts in open-web joists carrying about one bay, and upgraded three: small pipes through heavily loaded walls (16–35 bays) and a pipe embedded in a slab directly beneath a wall that has no other support (orphaned load 1.8 MN). This is the intended effect of separating local damage from global consequence.

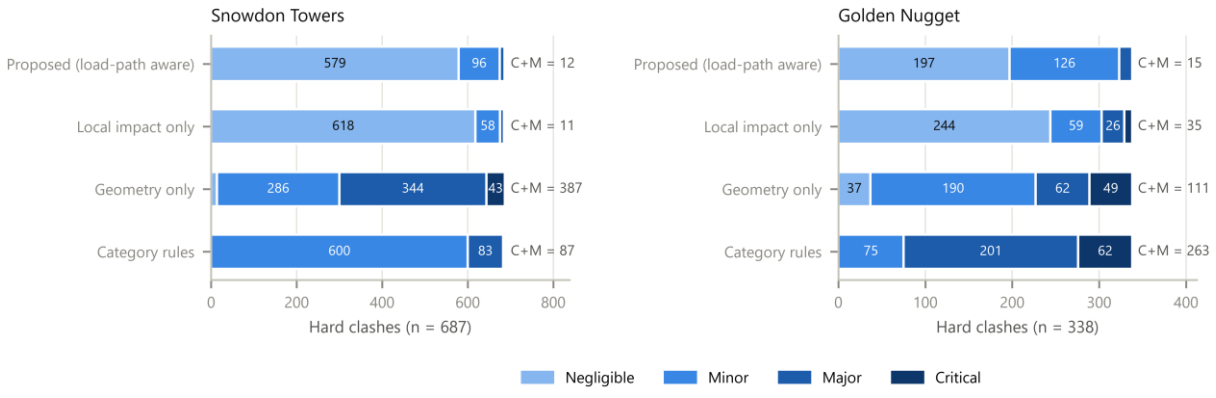


Fig. 4. Distribution of severity classes of the hard clashes under the four classification methods. "C+M" is the number of Critical and Major clashes.

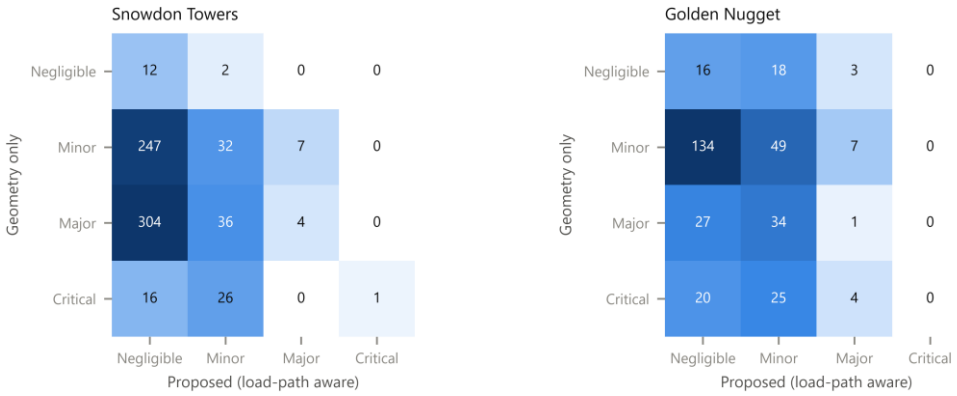


Fig. 5. Cross-tabulation of geometry-only and proposed classes for the hard clashes.

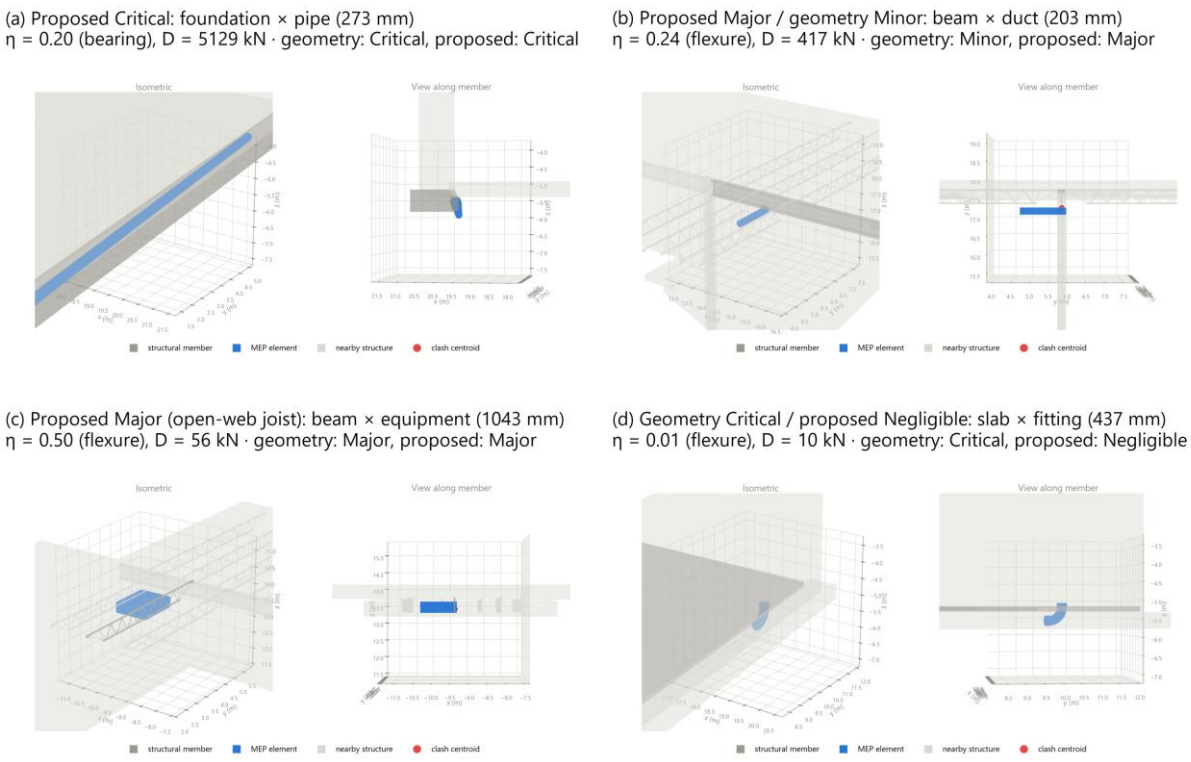


Fig. 6. Examples from Snowdon Towers (isometric view and view along the structural member). (a) A pipe running inside a heavily loaded foundation. (b) A duct clipping the bottom flange of a steel girder. (c) A mechanical unit cutting the chord zone of an open-web joist. (d) A large elbow in a slab on grade, Critical for geometry-only triage but structurally negligible.

5.4 Sensitivity (RQ3)

Fig. 7 summarises the one-at-a-time sensitivity analysis for both models. In Snowdon Towers the classification is insensitive to the consequence thresholds ($\kappa_w \geq 0.97$ for scaling between 0.5 and 2), to the live and superimposed dead loads ($\kappa_w \geq 0.99$), and to the contact and end tolerances ($\kappa_w = 1.00$). The orphan weight α and the score weight β do not change any class in either model, by construction, but they reorder the ranking within classes (Kendall's τ of the top 50 between 0.73 and 0.98). In both models the impact thresholds matter most. Scaling them by 0.67 increases the Critical+Major set from 12 to 19 in Snowdon Towers and from 15 to 30 in Golden Nugget, and scaling by 1.5 reduces it to 6 and 2 respectively (Jaccard 0.13–0.67, κ_w 0.76–0.94). Golden Nugget is also sensitive to two parameters that do not matter in Snowdon Towers. Halving the consequence thresholds raises its Critical+Major set to 28 (κ_w 0.62), because many of its walls carry one to three bay-equivalents, close to the C1/C2

boundary. Doubling the contact tolerance to 100 mm also gives 28 (κ_w 0.63), because more timber joints then count as supports and the unsupported set shrinks from 53 to 38. The impact thresholds, the consequence thresholds and, for timber framing, the contact tolerance are therefore the parameters that most need calibration against expert judgement (Section 6.2).

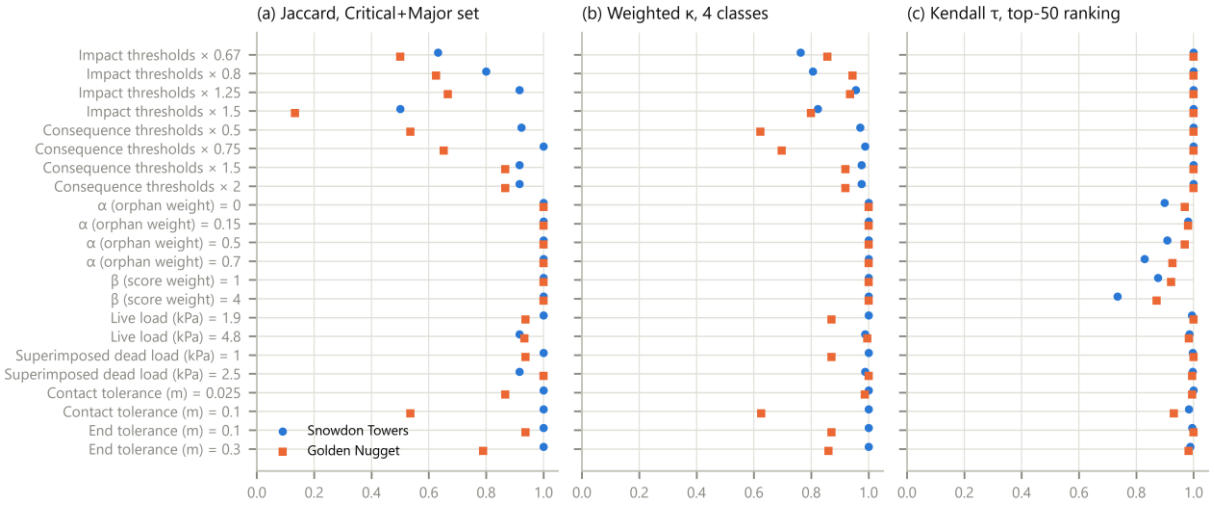


Fig. 7. One-at-a-time sensitivity of the severity classification (circles: Snowdon Towers; squares: Golden Nugget). (a) Jaccard overlap of the Critical+Major set, (b) quadratic-weighted κ of the four classes, and (c) Kendall's τ of the top-50 scores, each relative to the reference configuration.

5.5 Digital-twin performance (RQ4)

The initial full synchronisation of Snowdon Towers took 38 s. Over the stream of 120 design changes, the incremental update latency (Fig. 8) had a median of 62 ms for MEP changes (95th percentile 143 ms) and 5.6 s for structural changes (95th percentile 6.4 s). A full recomputation of the same states took 23–27 s (mean 24 s), so MEP changes were processed about 400 times faster. MEP changes require only a few narrow-phase tests, with a median of one candidate pair per event. Structural changes are dominated by re-propagating loads through the graph (median 2.4 s) and by re-assessing every clash (3.2 s), which is done conservatively because a change in support can alter slab and consequence contexts anywhere downstream. At every one of the six checkpoints, the incremental state was **identical** to the full recomputation: the same clash pairs, kinds, classes and η to six decimals. The twin also behaved sensibly: deleting MEP elements resolved 11 clashes, adding duplicated runs created

21, and lowering beams by 150 mm created 16 new clashes and resolved 4. In Golden Nugget the corresponding medians were 37 ms and 2.5 s, again with identical results at every checkpoint (Section 5.6).

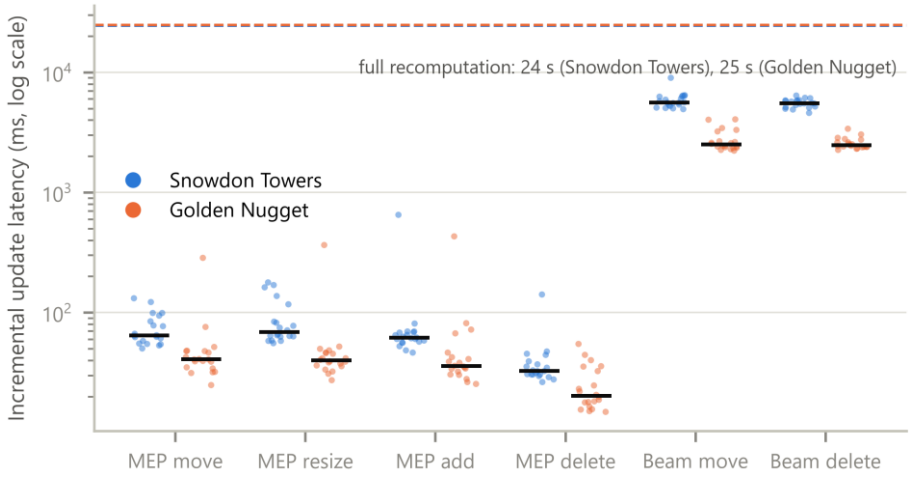


Fig. 8. Incremental update latency of the digital twin for 120 design changes per model; horizontal bars are medians, the dashed lines are the mean times of a full recomputation.

5.6 Held-out case: Golden Nugget

Golden Nugget is structurally unlike Snowdon Towers. Concrete walls, slabs and foundations carry a timber skeleton frame (posts with tenon joints, transoms, purlins and braces) together with steel HEB beams and columns. Its 637 structural members comprise 277 beams, 59 braces, 139 columns, 102 walls, 45 slabs and 15 foundations, and its building-services model contributes 5,283 of the 5,297 MEP elements (Table 3). The export of the building-services model was interrupted once and resumed: the resumable export keeps every complete record, so no element was exported twice.

Detection. The framework found 338 hard clashes, 31 touches and 715 clearance clashes. The clearance clashes are mostly on slabs (522) and walls (123). Against Revit's 357 reference pairs it achieved precision 0.962, recall 0.994 and F1 0.978 (Table 4). Of the 14 pairs not in Revit's set, 10 are touches and 4 are small overlaps (1.6–22.5 cm^3). The 2 pairs Revit reports but the framework does not are measured 11 and 20 mm apart by the framework, a discrepancy whose cause could not be resolved from the exported geometry. Detection took

46 s, longer than the 32 s Revit needed for its reference pairs: unlike Snowdon Towers, this model contains detailed equipment families whose dozens of components must be intersected one by one.

Determinism. The held-out run also exposed a reproducibility problem invisible in the development case. Equipment families with open components (for example an air-handling unit with 58 components and fan-coil units with 67) fell back to point sampling, and trimesh resolves inside/outside on open meshes with unseeded random rays. As a result, the twin's incremental state differed from a full recomputation in 14 of 1,079 clash values. Replacing sampling by convex-hull stand-ins for open components and a fixed-direction parity test (Section 3.3) made all results reproducible. It turned two touches into hard clashes and left detection agreement unchanged; 365 of the 369 physical clashes are now measured by an exact Boolean, 39 of them with hull stand-ins.

What the frozen method got wrong. With the method exactly as developed on Snowdon Towers, the load path conserved load exactly, but it exposed two assumptions that do not hold for this building (Table 6).

1. The reference bay load, then defined as the median beam load, collapsed to 0.3 kN. In this flat-slab building the floors bear directly on walls and columns, and the 277 framing members are mostly timber skeleton and secondary steel members that receive little floor load. Nearly every member therefore fell into the global consequence level, and 65 hard clashes (19 %) were rated Critical or Major, almost twice the 35 of the local-impact ablation.
2. Ninety members (14 %) had no path to the ground. Most were timber posts whose tenons sink about 100 mm into the sill beams they stand on, so the rule "a beam whose end reaches a column bears on it" fired first and reversed the support, and diagonal braces spanning several metres in height were treated as supports.

Revision, applied to both models. The reference bay load was redefined as the median load a member receives directly from a slab. This definition is independent of whether floors bear on joists, beams, walls or columns, and gives D_{ref} = 45.7 kN here and 51.0 kN (previously 54.4 kN) in Snowdon Towers. The beam–column rule now asks whether the vertical member continues below the beam, and braces are treated as lateral members that never support gravity load (Table 1). The unsupported set fell to 53 members (8.3 %): 28 timber beams (14 transoms, 13 hsbcad beams and 1 purlin), 18 slabs, 6 walls and 1 column, whose supports the rules still cannot infer from the modelled geometry. Nineteen edges were removed from two small cycles around a 50 mm steel column stub flush with the surrounding beams and slab. In Snowdon Towers the revision changed the unsupported set from 21 to 18 members and the class of three of the 687 hard clashes. Load was conserved exactly in both models (27.4 MN in Golden Nugget).

Table 6. Held-out case (Golden Nugget): load path and proposed severity of the hard clashes with the frozen method (as developed on Snowdon Towers) and after revising the reference bay load and the beam–column/brace rules (applied to both models).

Method	D_{ref} (kN)	Unsupported	Cycle edges removed	Crit. / Major / Minor / Negl.	Crit. + Major
Frozen	0.3	90 (14.1 %)	4	22 / 43 / 261 / 10	65 (19.3 %)
Revised	45.7	53 (8.3 %)	19	0 / 15 / 126 / 197	15 (4.4 %)

Severity. With the revised method, 15 of the 338 hard clashes (4.4 %) were rated Major, 126 Minor and 197 Negligible, with none Critical (Table 5). Geometry-only triage would prioritise 111 clashes (33 %) and category rules 263 (78 %), mostly because this model has many more column and wall clashes. Agreement with the proposed classes was again low (κ_w = 0.07 and 0.06). The load path changed the outcome more here than in Snowdon Towers: it downgraded 23 of the 35 Critical or Major clashes of the local-impact ablation and upgraded 3. All of the downgraded clashes lie in lightly loaded members, each carrying less than three bay-equivalents: 9 walls, 6 timber purlins, 5 slab panels and 3 braces. Timber members are assessed with the same rectangular-section mechanics as concrete, but without the

613 reinforcement-zone rule, which the first held-out run had applied to them wrongly. The
614 highest-ranked clashes are a 660 mm duct passing through a slab ($\eta = 1.0$), four ducts and
615 fittings cutting walls within their boundary zones (η = 0.42–0.48), and a pipe fitting at the
616 support of a timber purlin (shear, $\eta = 0.39$). The wall and brace downgrades rest on gravity
617 load only. If these members also resist lateral load, which the framework does not assess, they
618 may deserve more attention (Section 6.2).
619 **Digital twin.** In the twin benchmark the incremental state was again identical to the full
620 recomputation at all six checkpoints. MEP changes took a median of 37 ms (95th percentile
621 77 ms) and structural changes 2.5 s (95th percentile 3.5 s), against 23–29 s for a full
622 recomputation (Fig. 8). Structural updates are faster than in Snowdon Towers because the
623 graph is smaller and the severity re-assessment (0.6 s) covers fewer clashes. The initial
624 synchronisation took 60 s, because it also builds the meshes and collision structures of all
625 elements.

626 # 6 Discussion

627 ## 6.1 Implications for coordination practice

628 Across the two buildings, geometry-only triage would ask coordinators to prioritise 498 of
629 1,025 hard clashes (49 %) and category rules 350 (34 %). The proposed method prioritises 27
630 (2.6 %). The reduction is not a matter of setting thresholds more leniently. The disagreements
631 run in both directions: most clashes that geometry flags are routine sleeved penetrations of
632 slabs and walls, whereas several clashes it ranks Minor remove capacity where it matters,
633 such as a duct clipping a girder flange or a mechanical unit cutting a joist chord. Each class is
634 also explainable in engineering terms. It traces back to the governing action, the fraction of
635 capacity removed, the demand at that location, the load that depends on the member, the load
636 without an alternative path, and code-based flags such as “within 4h of column” or “opening >

h/3". That gives an engineer something to check rather than a score to trust. This complements relevance classifiers learned from past coordination decisions [3,4,6]: the local impact and consequence computed here could serve as structural features for such models, which at present see only element attributes.

The held-out case carries a broader lesson. Rules that look neutral can embed assumptions about the structural system: a "typical bay" measured by beams presumes a beam-and-slab building, and "a beam end at a column bears on the column" presumes steel or concrete framing rather than timber joinery with tenons. Both assumptions held in the development case and failed silently in the held-out one, and the failure surfaced only because the consequence distribution looked implausible. Automated structural reasoning on BIM data should therefore be tested across structural systems, not only across buildings, and its reference quantities should be defined by structural function (the load a member receives from the floor) rather than by element type.

For practice, incremental updates in tens of milliseconds for MEP edits and a few seconds for structural edits make it feasible to check clashes *while* modelling rather than in periodic coordination rounds. Together with the write-back of severity into the authoring view, this moves coordination towards the closed control loop that Sacks et al. describe for construction digital twins [11]. A by-product is a model-quality check: members without a load path (1.5 % and 8.3 % of the structural members here) point to missing supports or modelling conventions that would also trouble downstream structural analysis.

6.2 Validity and limitations

Severity has not yet been validated against expert judgement. The proposed classes rest on simplified section mechanics and code provisions, and the case studies show *how* they differ from geometry- and category-based triage, not that they are *correct*. The sensitivity analysis identifies the impact thresholds as the parameters that matter most. They should be

calibrated against expert ratings, and a blinded rating instrument with an analysis script is released for this purpose (Section 6.3). Until such a study is completed, the classes should be read as a transparent, reproducible prioritisation aid, not a substitute for an engineer's check.

The mechanics are deliberately simple. Demand shapes assume simply-supported members under uniform load, so continuity, concentrated loads and load redistribution are ignored. Capacities are expressed as fractions of the gross section, because reinforcement and connection details are rarely present in coordination models. Only gravity loads are considered, so the lateral role of walls and braces is not captured. This matters most for wall and brace penetrations: in the held-out case, 12 of the 23 Critical or Major clashes the load path downgraded were in walls or braces carrying little gravity load, and such members can be critical for lateral stability regardless. The wall boundary-zone flag is retained so that such clashes remain visible. These simplifications keep the analysis fast enough for a twin and independent of analysis models. A finite-element check of the highest-ranked clashes is a natural next step.

The load path inherits the quality of the model. Support relations are inferred from geometry, joins and member ends with fixed tolerances, and orientation rules encode typical gravity hierarchies. Members that are not connected to the ground are reported rather than guessed. In the development case they revealed genuine modelling gaps. In the held-out case 8.3 % of the members remained unsupported, mostly timber beams and slabs whose supports the rules cannot infer from the modelled joinery. For those members the consequence is underestimated, because their dependent load is limited to their own weight. The held-out case also showed that orientation rules carry assumptions about the structural system, so further systems (precast, cross-laminated timber, long-span trusses) may need additional rules. Whole-storey slabs require the local-panel approximation.

Detection is compared with Revit, not with ground truth. Revit's solid filter is an exact but independent implementation, and it excludes face contact, which is why touches account for the remaining false positives. Polygonal tessellation of curved MEP surfaces can create or remove sub-millimetre contacts.

The twin couples the authoring model, not the physical asset. In Kritzinger et al.'s terms [37], data flows automatically in both directions between the authoring model and the analysis. The physical building, however, is represented only through its model. The delta interface is agnostic to the source of a change, so as-built deviations from scan-to-BIM or construction-status updates could be ingested in the same way, but this was not tested here.

Generalisability. Two sample models of one authoring platform were analysed. They differ in structural system (a steel and joist frame with concrete cores; a concrete and timber hybrid), size and modelling conventions. The second was held out, but it was also used to revise two rules, so it can no longer serve as an unbiased test of the revised method. Further projects, especially real coordination datasets with documented decisions, are needed.

6.3 Future work

Calibrating and validating the severity model with expert ratings, using the released blinded instrument; FE-based verification of the top-ranked clashes; lateral load paths; ingesting physical-state data (scans, sensors, construction progress) through the same delta interface; and linking severity to resolution, for example by proposing the cheapest reroute that removes a Critical clash, are the most direct extensions.

7 Conclusions

This study presented a digital twin-enabled framework that detects structural clashes in federated Revit models and grades them by what they do to the structure. It was implemented

709 with Python and the Revit API and evaluated on two Revit 2027 sample buildings of different
710 structural systems. The main findings are:

711 1. Exact-mesh clash detection from Revit API data agreed closely with Revit's own
712 solid-intersection filter (F1 = 0.994 and 0.978), provided that full containment and
713 multi-solid families are handled explicitly. With an R-tree it scaled almost linearly to
714 82,000 elements.
715 2. A gravity load-path graph can be built automatically from BIM geometry, Revit joins
716 and member-end proximity, and it conserves load exactly. It flags members without a
717 modelled support (1.5 % and 8.3 %), which is itself useful model-quality information.
718 3. Combining a code-informed local capacity-loss index with load-path consequence
719 prioritised 27 of 1,025 hard clashes (2.6 %), compared with 49 % for geometry-only
720 triage. The two methods disagreed in both directions, which shows that penetration
721 depth alone is a poor proxy for structural severity.
722 4. In the development case the classes were robust to loads, tolerances and consequence
723 thresholds, but sensitive to the local-impact thresholds. In the held-out case the
724 consequence thresholds and the contact tolerance also mattered. These judgement-
725 based parameters therefore need calibration against expert judgement.
726 5. A held-out building exposed rules that embedded assumptions about the structural
727 system, and a reproducibility defect. The rules were revised to depend on structural
728 function rather than element type and the defect was removed; the revision changed
729 three of 687 classes in the development case.
730 6. As a digital twin, the analysis updated incrementally in tens of milliseconds for MEP
731 changes and a few seconds for structural changes, with results identical to full
732 recomputation, and wrote its assessment back into Revit.

The most important next step is a validation of the severity classes against blinded expert ratings, for which an instrument and analysis scripts are released with the code. Lateral load paths and the ingestion of as-built and sensor data through the same change interface would extend the framework from design coordination towards a twin of the physical structure.

Declarations

CRediT authorship contribution statement. Maleeha Waseem: Conceptualization, Methodology, Visualization, Writing – original draft, Writing – review & editing. **Muhammad Arham:** Data curation, Formal analysis, Validation, Writing – review & editing. **Muhammed Ahmed:** Conceptualization, Software, Investigation, Resources, Project administration, Writing – review & editing.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability. The source code (the pyRevit extension and the Python analysis package), the experiment scripts, the result tables and the expert rating instrument are openly available on GitHub at https://github.com/Darkiii-2514/DT-Clash-Detection-Paper. The case-study models are Autodesk Revit 2027 sample files that are installed with the software and are not redistributed. The raw model exports can be regenerated from them with the released extraction scripts.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work the authors used Claude Code with the Claude Opus 5.5 model (Anthropic) to search the literature, format the references and draft and revise the manuscript text, highlights and captions. The use of the same tool for software

development and data analysis is described in Section 4.1. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] R. Chahrour, M.A. Hafeez, A.M. Ahmad, H.I. Sulieman, H. Dawood, S. Rodriguez-Trejo, M. Kassem, K.K. Naji, N. Dawood, Cost-benefit analysis of BIM-enabled design clash detection and resolution, Construction Management and Economics 39 (2020) 55–72. https://doi.org/10.1080/01446193.2020.1802768.

[2] A. Akponeware, Z. Adamu, Clash detection or clash avoidance? An investigation into coordination problems in 3D BIM, Buildings 7 (2017) 75. https://doi.org/10.3390/buildings7030075.

[3] Y. Hu, D. Castro-Lacouture, Clash relevance prediction based on machine learning, Journal of Computing in Civil Engineering 33 (2019). https://doi.org/10.1061/(asce)cp.1943-5487.0000810.

[4] W.Y. Lin, Y.-H. Huang, Filtering of irrelevant clashes detected by BIM software using a hybrid method of rule-based reasoning and supervised machine learning, Applied Sciences 9 (2019) 5324. https://doi.org/10.3390/app9245324.

[5] A.S. Borkowski, A. Kubrat, A conceptual AI-based framework for clash triage in building information modeling (BIM): Towards automated prioritization in complex construction projects, Buildings 16 (2026) 690. https://doi.org/10.3390/buildings16040690.

[6] H.J. Koo, B.C. Guerra, S. Saka, F. Leite, Streamlining BIM coordination and clash resolution through risk and relevance analysis, Journal of Construction Engineering and Management 152 (2026). https://doi.org/10.1061/jcemd4.coeng-17676.

[7] R. Ailem, C. Boton, Automating clash relevance filtering in BIM-based multidisciplinary coordination using machine learning, Automation in Construction 181 (2026) 106644. https://doi.org/10.1016/j.autcon.2025.106644.

[8] I. Bitaraf, A. Salimpour, P. Elmi, A.A. Shirzadi Javid, Improved building information modeling based method for prioritizing clash detection in the building construction design phase, Buildings 14 (2024) 3611. https://doi.org/10.3390/buildings14113611.

[9] M. Grieves, J. Vickers, Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems, in: Transdisciplinary Perspectives on Complex Systems, Springer International Publishing, 2016: pp. 85–113. https://doi.org/10.1007/978-3-319-38756-7_4.

[10] C. Boje, A. Guerriero, S. Kubicki, Y. Rezgui, Towards a semantic construction digital twin: Directions for future research, Automation in Construction 114 (2020) 103179. https://doi.org/10.1016/j.autcon.2020.103179.

[11] R. Sacks, I. Brilakis, E. Pikas, H.S. Xie, M. Girolami, Construction with digital twin information systems, Data-Centric Engineering 1 (2020). https://doi.org/10.1017/dce.2020.16.

[12] D.-G.J. Opoku, S. Perera, R. Osei-Kyei, M. Rashidi, Digital twin application in the construction industry: A literature review, Journal of Building Engineering 40 (2021) 102726. https://doi.org/10.1016/j.jobe.2021.102726.

[13] U.S. Department of Defense, Design of buildings to resist progressive collapse (UFC 4-023-03, with change 3), Unified Facilities Criteria, Washington, DC, 2016.

[14] U. Starossek, M. Haberland, Disproportionate collapse: Terminology and procedures, Journal of Performance of Constructed Facilities 24 (2010) 519–528. https://doi.org/10.1061/(asce)cf.1943-5509.0000138.

[15] ACI Committee 318, Building code requirements for structural concrete (ACI 318-19) and commentary (ACI 318R-19), American Concrete Institute, Farmington Hills, MI, 2019.

[16] D. Darwin, Steel and composite beams with web openings, American Institute of Steel Construction, Chicago, IL, 1990.

[17] G. van den Bergen, Efficient collision detection of complex deformable models using AABB trees, Journal of Graphics Tools 2 (1997) 1–13. https://doi.org/10.1080/10867651.1997.10487480.

[18] S. Gottschalk, M.C. Lin, D. Manocha, OBBTree: A hierarchical structure for rapid interference detection, in: Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques, ACM, 1996: pp. 171–180. https://doi.org/10.1145/237170.237244.

[19] A. Guttman, R-trees: A dynamic index structure for spatial searching, ACM SIGMOD Record 14 (1984) 47–57. https://doi.org/10.1145/971697.602266.

[20] J. Pan, S. Chitta, D. Manocha, FCL: A general purpose library for collision and proximity queries, in: 2012 IEEE International Conference on Robotics and Automation, IEEE, 2012: pp. 3859–3866. https://doi.org/10.1109/icra.2012.6225337.

[21] Y. Hu, D. Castro-Lacouture, C.M. Eastman, Holistic clash detection improvement using a component dependent network in BIM projects, Automation in Construction 105 (2019) 102832. https://doi.org/10.1016/j.autcon.2019.102832.

[22] Y. Hu, D. Castro-Lacouture, C.M. Eastman, S.B. Navathe, Automatic clash correction sequence optimization using a clash dependency network, Automation in Construction 115 (2020) 103205. https://doi.org/10.1016/j.autcon.2020.103205.

[23] Y. Hu, D. Castro-Lacouture, C.M. Eastman, S.B. Navathe, Component change list prediction for BIM-based clash resolution from a graph perspective, Journal of Construction Engineering and Management 147 (2021). https://doi.org/10.1061/(asce)co.1943-7862.0002092.

[24] H.-C. Hsu, S. Chang, C.-C. Chen, I.-C. Wu, Knowledge-based system for resolving design clashes in building information models, Automation in Construction 110 (2020) 103001. https://doi.org/10.1016/j.autcon.2019.103001.

[25] L. Wang, F. Leite, Formalized knowledge representation for spatial conflict coordination of mechanical, electrical and plumbing (MEP) systems in new building projects, Automation in Construction 64 (2016) 20–26. https://doi.org/10.1016/j.autcon.2015.12.020.

[26] J. Liu, P. Liu, L. Feng, W. Wu, D. Li, Y.F. Chen, Automated clash resolution for reinforcement steel design in concrete frames via q-learning and building information modeling, Automation in Construction 112 (2020) 103062. https://doi.org/10.1016/j.autcon.2019.103062.

[27] W. Xu, P. Chen, H.-L. Chi, S. Wang, S. Zhou, Digital-physical integration for MEP-structural clash coordination using path planning and AR navigation, Automation in Construction 179 (2025) 106501. https://doi.org/10.1016/j.autcon.2025.106501.

[28] C. Eastman, J. Lee, Y. Jeong, J. Lee, Automatic rule-based checking of building designs, Automation in Construction 18 (2009) 1011–1033. https://doi.org/10.1016/j.autcon.2009.07.002.

[29] W. Solihin, C. Eastman, Classification of rules for automated BIM rule checking development, Automation in Construction 53 (2015) 69–82. https://doi.org/10.1016/j.autcon.2015.03.003.

[30] S. Zhang, J. Teizer, J.-K. Lee, C.M. Eastman, M. Venugopal, Building information modeling (BIM) and safety: Automatic safety checking of construction models and schedules, Automation in Construction 29 (2013) 183–195. https://doi.org/10.1016/j.autcon.2012.05.006.

[31] X. Xie, J. Zhou, X. Fu, R. Zhang, H. Zhu, Q. Bao, Automated rule checking for MEP systems based on BIM and KBMS, Buildings 12 (2022) 934. https://doi.org/10.3390/buildings12070934.

855 [32] I.J. Ramaji, A.M. Memari, Interpretation of structural analytical models from the
856 coordination view in building information models, Automation in Construction 90 (2018)
857 117–133. https://doi.org/10.1016/j.autcon.2018.02.025.
858 [33] R.M. Lawson, F. Hanus, D. Sonck, Large web openings in steel and composite beams,
859 Steel Construction 10 (2017) 168–175. https://doi.org/10.1002/stco.201710020.
860 [34] A. Ahmed, M.M. Fayyadh, S. Naganathan, K. Nasharuddin, Reinforced concrete
861 beams with web openings: A state of the art review, Materials &Amp; Design 40 (2012) 90–
862 102. https://doi.org/10.1016/j.matdes.2012.03.001.
863 [35] A.A. Elansary, A.A. Abdel Aty, H.A. Abdalla, M. Zawam, Shear behavior of
864 reinforced concrete beams with web opening near supports, Structures 37 (2022) 1033–1041.
865 https://doi.org/10.1016/j.istruc.2022.01.040.
866 [36] D. Mostofinejad, N. Jafarian, A. Naderi, A. Mostofinejad, M. Salehi, Effects of
867 openings on the punching shear strength of reinforced concrete slabs, Structures 25 (2020)
868 760–773. https://doi.org/10.1016/j.istruc.2020.03.061.
869 [37] W. Kritzinger, M. Karner, G. Traar, J. Henjes, W. Sihn, Digital twin in manufacturing:
870 A categorical literature review and classification, IFAC-PapersOnLine 51 (2018) 1016–1022.
871 https://doi.org/10.1016/j.ifacol.2018.08.474.
872 [38] C. Semeraro, M. Lezoche, H. Panetto, M. Dassisti, Digital twin paradigm: A
873 systematic literature review, Computers in Industry 130 (2021) 103469.
874 https://doi.org/10.1016/j.compind.2021.103469.
875 [39] M. Deng, C.C. Menassa, V.R. Kamat, From BIM to digital twins: A systematic review
876 of the evolution of intelligent building representations in the AEC-FM industry, Journal of
877 Information Technology in Construction 26 (2021) 58–83.
878 https://doi.org/10.36680/j.itcon.2021.005.

[40] X. Hu, G. Olgun, R.H. Assaad, An intelligent BIM-enabled digital twin framework for real-time structural health monitoring using wireless IoT sensing, digital signal processing, and structural analysis, Expert Systems with Applications 252 (2024) 124204. https://doi.org/10.1016/j.eswa.2024.124204.

[41] U. Starossek, M. Haberland, Approaches to measures of structural robustness, Structure and Infrastructure Engineering 7 (2011) 625–631. https://doi.org/10.1080/15732479.2010.501562.

[42] A. McKay, K. Marchand, M. Diaz, Alternate path method in progressive collapse analysis: Variation of dynamic and nonlinear load increase factors, Practice Periodical on Structural Design and Construction 17 (2012) 152–160. https://doi.org/10.1061/(asce)sc.1943-5576.0000126.

[43] S. Zhao, N. Wu, Q. Wang, Deep residual u-net with input of static structural responses for efficient u* load transfer path analysis, Advanced Engineering Informatics 46 (2020) 101184. https://doi.org/10.1016/j.aei.2020.101184.

[44] T. Lengauer, R.E. Tarjan, A fast algorithm for finding dominators in a flowgraph, ACM Transactions on Programming Languages and Systems 1 (1979) 121–141. https://doi.org/10.1145/357062.357071.

[45] American Society of Civil Engineers, Minimum design loads and associated criteria for buildings and other structures (ASCE/SEI 7-22), American Society of Civil Engineers, Reston, VA, 2022. https://doi.org/10.1061/9780784415788.

[46] A.A. Hagberg, D.A. Schult, P.J. Swart, Exploring network structure, dynamics, and function using NetworkX, in: Proceedings of the 7th Python in Science Conference (SciPy 2008), 2008: pp. 11–15.

[47] pyRevitLabs, pyRevit: Rapid application development (RAD) environment for autodesk revit, (2026).

[48] Autodesk, Revit 2027 API (RevitAPI.dll, RevitAPIUI.dll), (2026).